

SOFTWARE REQUIREMENTS SPECIFICATION

Milestone 1 — Project Initialization, Architecture & Core Setup

SwipeX

Swipe-Based Intelligent Job Discovery and Career Assistance Platform

Field	Detail
Document Type	Software Requirements Specification (SRS) — Milestone 1
Milestone Duration	7 Working Days
Team Size	5 Interns, working in parallel across five domains
Team Composition	Team Lead / Backend & Identity Management; Job & Data Intelligence; AI/ML Career Intelligence; Frontend & User Experience; Platform, Testing & Deployment
Deployment Scope	Excluded from Milestone 1. Production deployment occurs only after Milestone 4.
Tooling Policy	Every tool used must be free / open-source — no paid subscriptions required anywhere in this milestone
Primary Reference	SwipeX Project Charter (provided PDF) and supervising discussions
Prepared For	Educator / Mentor supervising the intern cohort

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the scope, tasks, responsibilities, dependencies, and integration requirements for Milestone 1 of the SwipeX platform. It translates the project charter into a coordinated, day-wise execution plan for five interns working in parallel, each owning a distinct technical domain, so that their individual deliverables converge into a single, functioning system increment by the end of Day 7.

1.2 Scope of Milestone 1

Milestone 1 covers project initialization, system architecture, and core setup. By the end of Day 7 the team must deliver a working increment consisting of: an initialized multi-service repository with an agreed architecture; a functioning authentication and role-based access control (RBAC) system supporting Job Seeker, Recruiter, and Admin roles; a job/company data schema with a live Job & Swipe Service; a resume-upload flow backed by a heuristic ATS-scoring and recommendation stub that unblocks downstream integration ahead of real NLP/ML model training in Milestone 2; a connected frontend exposing login, swipe-based job discovery, and a dashboard shell; and a validated, tested integration of all of the above inside a local/dev Docker Compose environment. Production deployment is explicitly out of scope for Milestone 1 and is deferred until all four milestones are complete.

1.3 Definitions & Abbreviations

- SRS — Software Requirements Specification
- RBAC — Role-Based Access Control
- JWT — JSON Web Token, used for stateless authentication
- ATS — Applicant Tracking System (resume/job compatibility scoring)
- API Gateway — Single entry point that routes requests to backend services
- Stub / Heuristic Service — A simplified, rule-based placeholder for a component (e.g., ATS scoring, recommendations) whose full implementation is scheduled for a later milestone
- M1–M4 — Milestone 1 through Milestone 4, as defined in the project charter
- Intern 1–5 — Shorthand for the five domain owners defined in Section 4

1.4 Reference Documents

- SwipeX: Swipe-Based Intelligent Job Discovery and Career Assistance Platform — Project Charter (source PDF), Sections 1–9
- Architecture Diagram — SwipeX System Architecture (source PDF, Section 3)
- Prior mentor/team discussions establishing the five-domain team split and the deployment exclusion for Milestone 1

1.5 Golden Rule for This Milestone

Every tool used in Milestone 1 must be free. No intern should need to pay for anything — free code editors, free version control, free API-testing tools, free local container tooling, free CI minutes, free linting/security tools, and free ML/NLP libraries are sufficient to complete everything described in this document.

2. Free Tools Checklist (No Paid Tool Required Anywhere)

Before Day 1 begins, make sure every intern has access to the free tools below that are relevant to their domain.

What It's Needed For	Free Tool Option(s)
Writing code	VS Code (completely free)

What It's Needed For	Free Tool Option(s)
Saving & sharing code	Git + GitHub (free for public / small private repos)
Testing APIs manually	Postman (free plan) or Thunder Client extension in VS Code (free)
Backend framework	Python + FastAPI (free) or Django REST Framework (free)
Frontend framework	React.js (free) with Tailwind CSS (free)
Authentication (JWT / OAuth2)	PyJWT / python-jose (free); Authlib for OAuth2 (free)
Database	PostgreSQL (free) for structured data; Redis (free) for caching / sessions
Automated backend tests	Pytest (free) or Django Test Client (free)
Automated frontend tests	Jest + React Testing Library (free)
Automated local integration tests	Docker Compose (free) — full stack on one laptop, no live server needed
CI checks on every PR (lint / build / test only)	GitHub Actions (free tier) — no deployment stage configured
Code quality / linting	ESLint (frontend, free), Flake8 / Black (backend, free)
API documentation	Swagger UI / OpenAPI (free, built into FastAPI / DRF)
Resume parsing & NLP experimentation (heuristic stub only in M1)	Python, pdfplumber / PyPDF2, spaCy, scikit-learn (all free); Google Colab for extra free compute
Diagrams (architecture, ER diagram)	draw.io / diagrams.net (free)
Wireframing / UI design	Figma (free tier)
Defect tracking / task board	GitHub Issues & Projects (free) or Trello (free tier)

None of the tools above require a paid plan to complete Milestone 1. If a paid tier is ever tempting (e.g., a hosted DB add-on), the local/free alternative listed here is sufficient for this milestone's scope.

3. Overall Description

SwipeX is a centralized platform that helps job seekers discover and apply for relevant jobs through an intuitive swipe-based interface, backed by AI-powered resume analysis, ATS scoring, personalized recommendations, and real-time opportunity tracking across MNCs, startups, and newly founded companies. The architecture follows five layers: Frontend, API Gateway, Backend Services (Job, Swipe, Matching, Resume, ATS Scoring, AI Recommendation, Application, Notification, Analytics), AI/NLP Services, and the Database & Storage layer (PostgreSQL, Redis, file storage, vector database), sitting on top of a containerized infrastructure. Milestone 1 establishes the foundation of this stack — the entry points, the data backbone, and the first three backend services (Authentication, Job/Swipe, and a heuristic Resume/ATS/Recommendation stub) — while stubbing the deeper AI logic so all layers can be integrated end-to-end without waiting for full model development.

4. Team Structure & Domain Ownership

Intern	Domain	Primary Architecture Ownership (Milestone 1)
Intern 1	Team Lead / Backend & Identity Management	API Gateway, Authentication Service (JWT + OAuth2), RBAC, User & Role Management Service, repository structure, service integration, API contracts
Intern 2	Job & Data Intelligence	Job / company / startup data schema, seed dataset, Job Listing Service, Swipe Service (interaction schema + APIs), filtering & search logic

Intern	Domain	Primary Architecture Ownership (Milestone 1)
Intern 3	AI/ML Career Intelligence	Resume Service skeleton, feature planning, EDA on resumes & job descriptions, heuristic ATS-scoring stub, heuristic recommendation stub, Milestone-2 ML roadmap
Intern 4	Frontend & User Experience	React application, UI/UX wireframes, login UI, swipe-based job-discovery UI, dashboard shell, resume-upload UI
Intern 5	Platform, Testing & Deployment	Code-quality tooling, CI (build/lint/test — no deploy), local Docker Compose environment, test design & automation, defect tracking, integration validation

Deployment (packaging for production, cloud release, AWS/Azure hosting) is intentionally excluded from Intern 5's Milestone 1 scope. It is scheduled after Milestone 4 is complete.

5. Milestone 1 Objectives

- Finalize system architecture, database schema, and API contracts across all services.
- Stand up project repositories, coding standards, and a local multi-service Docker Compose environment for integration (development only, not production).
- Implement authentication and role-based access control (Job Seeker, Recruiter, Admin).
- Implement job/company/startup data schema and a live Job & Swipe Service: listings, filtering/search, and swipe-action recording.
- Stand up a Resume Service and a heuristic ATS-scoring / recommendation stub to unblock frontend/QA integration ahead of real NLP model training in Milestone 2.
- Deliver a connected frontend with login, a swipe-based job-discovery interface, and a dashboard shell consuming real backend APIs.
- Validate the full increment through integration testing, automated checks, and a defect-tracked QA cycle — without production deployment.

6. Functional Requirements — Milestone 1 Scope

Req ID	Requirement	Owner	Priority
FR-01	Users can register / log in (email-password and OAuth2) and receive a JWT encoding their role	Intern 1	High
FR-02	Protected endpoints enforce RBAC (Job Seeker / Recruiter / Admin)	Intern 1	High
FR-03	System stores job, company and startup records per a defined schema and exposes seed data	Intern 2	High
FR-04	System exposes job-listing APIs with filter / search (company type, location, remote, salary, skills)	Intern 2	High
FR-05	System records swipe actions (right = apply/save, left = skip) and per-user swipe history	Intern 2	High
FR-06	System exposes a resume-upload endpoint that stores and parses basic resume fields	Intern 3	Medium
FR-07	System exposes a heuristic ATS-score endpoint comparing a resume to a job description (placeholder for the Milestone-2 NLP model)	Intern 3	Medium
FR-08	System exposes a heuristic job-recommendation endpoint based on skill / keyword overlap	Intern 3	Medium

Req ID	Requirement	Owner	Priority
FR-09	Users can log in through the web application and reach a role-aware landing screen	Intern 4	High
FR-10	Users can browse jobs through a swipe-based interface consuming live Job Service data	Intern 4	High
FR-11	Dashboard displays saved/applied jobs and, where available, ATS-score / recommendation stub output	Intern 4	High
FR-12	All services run together inside a local Docker Compose integration environment	Intern 5	High
FR-13	All pull requests pass automated lint / build / test checks before merge	Intern 5	High
FR-14	The Milestone 1 increment passes a documented integration and RBAC test cycle	Intern 5	High

7. Interface Contracts & Integration Points

These contracts are frozen by Day 2 so that all five interns can build against a stable interface instead of waiting on one another's implementation.

7.1 Authentication & User Service (owner: Intern 1)

- POST /api/v1/auth/register — { name, email, password, role } → 201 { user_id }
- POST /api/v1/auth/login — { email, password } → 200 { access_token, refresh_token, role }
- GET /api/v1/auth/oauth/google — OAuth2 login → 200 { access_token, role }
- POST /api/v1/auth/refresh — { refresh_token } → 200 { access_token }
- GET /api/v1/auth/me — Header: Bearer token → 200 { user_id, name, role }

Consumed by: Intern 4 (login UI), Intern 5 (auth test suite), and indirectly by Intern 2 & 3 endpoints that require role-gated access.

7.2 Job & Swipe Service (owner: Intern 2)

- GET /api/v1/jobs?type=&location=&remote=&salary_min=&salary_max=&skills= → 200 [{ job_id, title, company, type, location, salary_range, skills[], posted_at }]
- GET /api/v1/jobs/{job_id} → 200 job detail
- POST /api/v1/swipes — { job_id, direction } → 201 { swipe_id }
- GET /api/v1/swipes/history → 200 [{ job_id, direction, swiped_at }]

Consumed by: Intern 4 (swipe UI / dashboard), Intern 3 (feature input for the recommendation stub), Intern 5 (functional + regression tests).

7.3 Resume & Career Intelligence Service — Heuristic Stub (owner: Intern 3)

- POST /api/v1/resume/upload (multipart file) → 201 { resume_id, parsed_skills[] }
- GET /api/v1/resume/{resume_id}/ats-score?job_id={id} → 200 { match_score, missing_keywords[], generated_by: "heuristic-v0" }
- GET /api/v1/recommendations?user_id={id} → 200 [{ job_id, match_percentage, generated_by: "heuristic-v0" }]

These endpoints use simple rule-based logic (keyword/skill overlap) rather than a trained model. They exist solely to give Intern 1 (gateway routing), Intern 4 (UI hook-up), and Intern 5 (test coverage) a stable contract to integrate against before real NLP/embedding-model training begins in Milestone 2.

7.4 Local Integration Environment (owner: Intern 5)

- docker-compose.dev.yml — brings up PostgreSQL, Redis, and all three backend services together for local integration testing
- Shared Postman/pytest collection — used by all interns to validate their endpoints against the frozen contracts above

This environment is for local/dev integration validation only and is explicitly not a deployment target.

8. Cross-Team Dependency Matrix

Consumer	Depends On	Artifact Needed	Needed By
Intern 1	Intern 5	Local DB / Redis environment (docker-compose skeleton)	Day 2
Intern 2	Intern 1	Approved DB technology & schema conventions	Day 1
Intern 3	Intern 2	Job / company data schema + seed dataset	Day 2
Intern 3	Intern 1	Auth contract (role claims used for personalisation)	Day 3
Intern 4	Intern 1	Auth API contract + working /login, /register	Day 3
Intern 4	Intern 2	Job & Swipe API	Day 4
Intern 4	Intern 3	ATS-score & recommendation stub API	Day 5
Intern 5	Intern 1 / 2 / 3	Dev endpoints for each service as they land	Days 3–5
Intern 1	Intern 2 / 3	Service routes to register in the API Gateway	Days 4–5

9. Master 7-Day Execution Timeline

All five interns work in parallel every day. The table below is the single-glance view; Section 10 gives the full detail (task, deliverable, dependency, hand-off) for each intern.

Day	Intern 1 Backend & Identity	Intern 2 Job & Data Intelligence	Intern 3 AI/ML Career Intelligence	Intern 4 Frontend & UX	Intern 5 Platform/Testing
1	Architecture + repo + API contract skeleton	Job/company data schema + synthetic dataset	ML/NLP env setup + feature planning	Wireframes + React init	Lint/CI standards + test plan + defect tracker
2	DB schema + backend skeleton + JWT/OAuth2 design	Job/Swipe schema + seed-data script	EDA on job & resume samples	Components + routing + API client	CI (lint/build) + docker-compose.dev.yml
3	Auth APIs + RBAC middleware	Job Service APIs (/jobs, /jobs/{id})	Resume Service skeleton + upload/parse stub	Login/Register UI wired to Auth API	Test cases for Auth + Job Service; API test skeleton
4	RBAC enforcement + gateway routing to Job Service	Swipe Service APIs + filter/search logic	Heuristic ATS-scoring logic on live data	Swipe interface (job cards) wired to Job API	Execute functional tests; CI runs tests on PR
5	Integrate Job/Swipe + Resume/ATS behind Gateway	Support integration; document data feed for M2	Integrate ATS + recommendation stub; M2 roadmap doc	Dashboard shell + resume upload + ATS display	Full integration test cycle (Auth+Job+ATS+UI)

Day	Intern 1 Backend & Identity	Intern 2 Job & Data Intelligence	Intern 3 AI/ML Career Intelligence	Intern 4 Frontend & UX	Intern 5 Platform/Testing
6	End-to-end backend integration + bug fixes	Unit tests + query optimisation + defect fixes	Unit tests for heuristic logic + defect fixes	UI polish, responsiveness, error handling	Regression testing + static code-quality reports
7	Code freeze + Swagger docs + sign-off	Final validation + module handoff notes	Final review + M1 scope / M2 handoff doc	Final UI fixes + demo walkthrough prep	Final QA sign-off report + M1 test summary

10. Detailed Day-Wise Task Breakdown (Per Intern)

For every day, the table explains the task in simple terms, lists a separate checklist to tick off once the work is actually done, and shows what the intern depends on to start that task and what they hand off to the rest of the team. Tick each checklist box only when that item is actually working, not just attempted.

Intern 1 — Backend & Identity Management

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
1	Sit with the team and lock the overall architecture. Create the shared code repository with folders for each service — like drawing the floor plan before building starts. Write a simple first draft of what the Auth, Job/Swipe, and Resume/ATS APIs will look like.	☐ Architecture finalized ☐ Repo created with service folders ☐ Draft API contracts written ☐ Plan shared with team	None	All interns (repo, contracts)
2	Design the database tables for Users and Roles. Build a basic skeleton of the backend app that will later route requests to every other service. Decide how login tokens (JWT) and "Sign in with Google" (OAuth2) will work.	☐ Users/Roles schema finalized ☐ Backend skeleton running ☐ JWT/OAuth2 approach decided ☐ Local DB/Redis confirmed working	Intern 5's local DB/Redis environment	Intern 2/3 (gateway pattern to plug into)
3	Build the real login system: Register, Login, Sign-in-with-Google, Refresh Token, and "who am I". Add a rule that decides what each type of user (Job Seeker, Recruiter, Admin) is allowed to do.	☐ Register API working ☐ Login + OAuth2 working ☐ Refresh API working ☐ Role rule (RBAC) added & tested	Own DB schema	Intern 4 (login integration), Intern 5 (test target)
4	Make sure the login rules are enforced everywhere. Connect the shared entry point (Gateway) to Intern 2's Job/Swipe service so requests pass through smoothly. Leave a placeholder connection ready for Intern 3's Resume/ATS service.	☐ RBAC enforced on protected routes ☐ Gateway routes to Job Service ☐ Placeholder route for Resume/ATS ☐ Tested with different user roles	Intern 2's job endpoints (skeleton)	Intern 4 (protected dashboard routes)
5	Bring the Resume/ATS stub online through the same Gateway, so Auth + Job/Swipe + Resume/ATS are all reachable from one place today. Update the shared Postman collection so the whole team can test easily.	☐ All three services reachable via one Gateway ☐ Postman collection updated ☐ Joint test with Intern 5 done	Intern 2 & 3's completed endpoints	Intern 4 & 5 (integrated backend to consume/test)
6	Run the whole backend together, fix any issues QA (Intern 5) finds, and write simple documentation (Swagger) so anyone can see what each API does.	☐ End-to-end backend test done ☐ QA-reported bugs fixed ☐ Swagger/API docs updated	Intern 5's defect log	Intern 5 (fixed build for regression)

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
		☐ Backend confirmed stable		
7	Final Integration Day — freeze the backend code, take part in the full team demo, and write a short summary of what was built.	☐ Backend code frozen ☐ Took part in full team demo ☐ Short backend summary written ☐ Last-minute critical bugs fixed	Passed QA regression	Milestone 1 demo / mentor review

Intern 2 — Job & Data Intelligence

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
1	Decide how job, company, and startup information will be organized (title, skills, salary, company type) and create a sample dataset that pretends to be real job postings — like designing a form before collecting real entries. Confirm the database choice with Intern 1.	☐ Job/company data design finalized ☐ Sample job/company dataset created ☐ Database tech confirmed with Intern 1	None (runs in parallel)	Intern 3 (schema for feature planning)
2	Finalize how a "swipe" is recorded (which job, which direction, when), and write a small script that fills the database with sample jobs and swipes so testing can begin.	☐ Job/Swipe schema finalized ☐ Seed script working ☐ Dev database populated	Own schema; Intern 5's DB container	Intern 3 (EDA data source)
3	Build the APIs that let other parts of the system ask: "show me the jobs" and "show me details of this one job", including basic filters like location, remote, and salary.	☐ Job-listing API working ☐ Job-detail API working ☐ Filters (type/location/remote/salary/skills) working	Own seed data	Intern 1 (gateway registration), Intern 4 (swipe UI data), Intern 5 (test target)
4	Build the swipe feature: save a swipe when a user swipes right/left, and let a user see their swipe history. Add a fast cache (Redis) so the job feed loads quickly.	☐ Swipe-save API working ☐ Swipe-history API working ☐ Redis caching added	Intern 5's Redis service	Intern 4 (richer dashboard data), Intern 3 (swipe-behaviour input for M2)
5	Connect the Job/Swipe service into the shared Gateway with Intern 1, and write a short note explaining the job/swipe data so Intern 3 can use it for real recommendations in Milestone 2.	☐ Gateway integration verified ☐ M2 data-feed notes documented	Intern 1's gateway routing	Intern 3 (M2 planning input)

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
6	Write simple automated tests for the job-listing and swipe logic, speed up any slow database queries the team points out, and fix bugs QA reports.	☐ Unit tests written ☐ Slow queries fixed ☐ QA-reported bugs fixed	Intern 5's defect log	Intern 5 (regression build)
7	Final Integration Day — confirm the Job/Swipe service works smoothly with everyone else's work, and write a short handover note for Milestone 2.	☐ Final test with full team done ☐ Module docs and handoff notes written	Passed regression	Milestone 1 demo

Intern 3 — AI/ML Career Intelligence

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
1	Set up the AI/NLP tools (pandas, scikit-learn, spaCy, pdfplumber) and think through what information (skills, keywords, experience) would help score a resume against a job — like deciding what clues a detective would look for.	☐ ML/NLP environment ready ☐ Feature plan written down ☐ Plan shared with the team	None (uses provisional mock data)	Self (Day 2 EDA)
2	Once Intern 2's sample job data is ready, explore it and a few sample resumes to look for useful patterns — this is called Exploratory Data Analysis (EDA).	☐ Job/resume data explored ☐ Findings written down ☐ Key features confirmed	Intern 2's dataset/schema	Own Day-3 service design
3	Build a simple Resume Service that lets a user upload a resume file and pulls out basic information (text, skill keywords) using a simple, non-AI parser for now.	☐ Resume Service skeleton created ☐ Upload endpoint working ☐ Basic skill extraction working	Own feature plan	Intern 1 (route to register), Intern 4 (contract to build against)
4	Turn the placeholder into simple rule-based scoring — for example, "count how many required skills from the job also appear in the resume" — and use the same idea to suggest matching jobs.	☐ ATS-score endpoint working ☐ Recommendation endpoint working ☐ Response format finalized	Intern 2's live job data	Intern 1 (integration), Intern 4 (UI hook-up)
5	Connect the Resume/ATS/Recommendation service into the shared Gateway with Intern 1, agree with Intern 4 on how scores will be shown on the dashboard, and write a short plan for the real AI model in Milestone 2.	☐ Services integrated behind Gateway ☐ M2 roadmap written ☐ Display plan agreed with Intern 4	Intern 1's gateway	Intern 4 (display contract), Mentor (M2 plan)
6	Write simple tests for the rule-based scoring and recommendation logic, and fix anything QA finds broken.	☐ Tests written for scoring/recommendation logic	Intern 5's defect log	Intern 5 (regression build)

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
		☐ QA-reported bugs fixed		
7	Final Integration Day — join the full team test, and clearly write down that this milestone only used simple rules, not a trained AI model.	☐ Final test with full team done ☐ Scope documented as heuristic-only ☐ Handover notes written	Passed regression	Milestone 1 demo

Intern 4 — Frontend & User Experience

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
1	Set up the website project and sketch simple designs for the Login, Swipe/Job-Discovery, Dashboard, Resume-Upload, and Profile screens — like drawing a floor plan before building a house.	☐ React project created ☐ Wireframes made for all key screens ☐ Plan shared with the team	Intern 1's draft API contracts (for screen data needs)	Self (Day 2 build)
2	Build the reusable building blocks of the website — navigation bar, layout, job card, buttons — and set up the basic pages and navigation between them.	☐ Reusable components built ☐ Page routing set up ☐ API client configured	Own wireframes	Self (Day 3 auth integration)
3	Build the Login and Register pages and connect them to Intern 1's real login system, including a "Sign in with Google" button, so a real user can actually log in.	☐ Login page working ☐ Register page working ☐ OAuth2 (Google) button wired	Intern 1's Auth API	Intern 5 (UI test target)
4	Build the swipe-based job-discovery screen — job cards that a user can swipe right (apply/save) or left (skip) — using real job data from Intern 2's service.	☐ Job cards show live data ☐ Swipe right/left saved via API ☐ Basic filters wired up	Intern 2's Job/Swipe API	Intern 5 (swipe-flow test target)
5	Build the Dashboard (saved and applied jobs) and the Resume-Upload screen, and show Intern 3's resume score and job recommendations on the dashboard.	☐ Dashboard shows saved/applied jobs ☐ Resume upload working ☐ ATS score/recommendations displayed	Intern 2's data; Intern 3's stub APIs	Intern 5 (full UI test target)
6	Polish the app — fix rough spots, make sure it works on different screen sizes, and show friendly messages if something goes wrong instead of a blank/broken screen.	☐ Responsive design checked ☐ Friendly error messages added ☐ QA-reported bugs fixed	Intern 5's defect log	Intern 5 (regression build)
7	Final Integration Day — fix any last issues and rehearse a simple walkthrough/demo of the whole app.	☐ Final test with full team done ☐ Demo walkthrough rehearsed ☐ Frontend notes documented	Passed regression	Milestone 1 demo

Intern 5 — Platform, Testing & Deployment

Day	Task Expected To Be Completed	Checklist	Depends On	Provides To
1	Set up the ground rules for good code (linting tools) and create a simple checklist/board to track testing tasks and bugs for Milestone 1 — like preparing a quality-control checklist before a factory line starts.	☐ Linting/code-style tools set up ☐ Test plan document created ☐ Bug/defect tracking board set up	Intern 1's repo structure	All interns (quality gates)
2	Set up an automatic check (CI) that runs every time someone submits code, to make sure it builds and follows the rules. Set up a way for the whole team to run all services together on one laptop using Docker, for local testing only.	☐ CI pipeline set up (lint + build only) ☐ docker-compose file created ☐ Confirmed full stack runs locally	Intern 1's backend skeleton	Intern 1/2/3 (shared dev environment)
3	Write down, in a simple list, what needs to be tested for Login and the Job/Swipe service, and set up a tool to run those tests automatically.	☐ Test cases written for Auth & Job/Swipe ☐ Automated testing tool set up	Intern 1 & 2's API contracts	Self (Day 4 execution)
4	Actually run those tests against Intern 1 and Intern 2's real APIs, note down anything broken with a priority, and make the automatic check also run tests every time.	☐ Auth & Job/Swipe APIs tested ☐ Bugs logged with priority ☐ CI now also runs automated tests	Intern 1 & 2's live endpoints	Intern 1 & 2 (defect list to fix)
5	Bring everyone's work together on one shared local setup and test the whole journey — login, browse/swipe jobs, upload a resume, see a score — and note down anything that doesn't work.	☐ Full local integration test run done ☐ Role-based access tested end-to-end ☐ All issues found are logged	All four other interns' Day-5 integrated build	All interns (defect backlog for Day 6)
6	Re-test everything after the team's fixes, and run a basic code-quality check to catch messy or risky code.	☐ Regression testing done after fixes ☐ Code-quality report generated ☐ System confirmed stable	Fixes from Interns 1–4	All interns (go/no-go signal)
7	Final Integration Day — run one last full check of everything, write a simple summary of what was tested and what passed, and confirm no deployment work has been done (that comes later).	☐ Final full test cycle completed ☐ QA sign-off report written ☐ Confirmed: no deployment activity performed	Clean regression build	Mentor / Milestone 1 review

11. Daily Coordination & Integration Sync Protocol

11.1 Daily Standup (15 minutes, start of day)

- Each intern states: what was completed yesterday, what is planned today, and any blocker relative to the Dependency Matrix (Section 8).
- Intern 1 (Team Lead) tracks blockers and re-sequences work if a dependency is late.

11.2 Integration Checkpoints

Checkpoint	Day	Purpose
Contract Freeze	Day 2 (EOD)	Auth, Job/Swipe, and Resume/ATS API contracts are frozen; no breaking changes without team sign-off
First Vertical Slice	Day 4	Auth + Job/Swipe reachable end-to-end from the frontend through the Gateway
Full Integration	Day 5	All four services (Auth, Job/Swipe, Resume/ATS stub, Frontend) run together in docker-compose.dev.yml and pass smoke tests
Regression & Sign-off	Day 7	Final validation cycle passes; Milestone 1 increment is signed off by QA and the Team Lead

11.3 Change Control

Any change to a frozen interface contract after Day 2 must be raised by the owning intern in standup, agreed by all consumers listed in Section 8, and reflected the same day in the shared Postman/pytest collection maintained by Intern 5.

12. Milestone 1 Deliverables Checklist

Deliverable	Owner	Due
Architecture document + repository structure	Intern 1	Day 1
Frozen API contracts (Auth, Job/Swipe, Resume/ATS)	Intern 1 / 2 / 3	Day 2
DB schema (Users/Roles + Job/Company/Swipe data)	Intern 1 / 2	Day 2
docker-compose.dev.yml + CI (lint/build/test)	Intern 5	Day 2 / 4
Working Auth + RBAC service (JWT + OAuth2)	Intern 1	Day 3–4
Working Job & Swipe Service (listing, filter, swipe, history)	Intern 2	Day 3–4
Resume Service + heuristic ATS-score & recommendation stub	Intern 3	Day 3–4
Login page + Swipe-based Job Discovery UI + Dashboard shell	Intern 4	Day 3–5
Test plan, test cases, defect log	Intern 5	Day 1–7 (continuous)
Full integration inside local Docker Compose	All (led by Intern 1 & 5)	Day 5
Regression report + code-quality report	Intern 5	Day 6
Milestone 1 sign-off report + demo	All	Day 7

13. Milestone 1 Acceptance Criteria

- Project initialization, architecture, and database schema are complete and documented.
- Authentication and role-based access control (Job Seeker / Recruiter / Admin) are implemented and enforced across protected routes.

- Job/company/startup data schema and a live Job & Swipe Service (listing, filter/search, swipe recording) are functional and expose data via REST APIs.
- The frontend presents a working login flow, a swipe-based job-discovery interface, and a dashboard shell driven by real backend data.
- The Resume Service and heuristic ATS-score/recommendation stub are reachable end-to-end through the Gateway and displayed on the dashboard, clearly scoped as a placeholder for Milestone 2's trained model.
- All services run together and pass integration/smoke tests inside the local Docker Compose environment.
- CI enforces lint, build, and automated test checks on every pull request.
- Every tool used across the milestone was free/open-source — no paid subscriptions were required.
- No production deployment activity has occurred; deployment remains explicitly scheduled for after Milestone 4.

14. Risks & Mitigations

Risk	Impact	Mitigation
Interface contracts change after Day 2, breaking downstream work	High	Contract-freeze checkpoint (Section 11.2) + change-control process (Section 11.3)
Auth service delayed, blocking Frontend and QA	High	Frontend/QA build against a mocked Auth response until Day 3; Intern 1 prioritizes Auth over gateway polish
Synthetic job/resume data unrepresentative, limiting later ML value	Medium	Intern 2 documents data-generation assumptions; Intern 3 flags gaps early in the M2 roadmap
Scope creep — AI/ML intern attempts real NLP/model training in M1	Medium	Explicit scope boundary in Sections 10 & 13: heuristic stub only in Milestone 1
Local integration environment differs across teammates' machines	Medium	Single shared docker-compose.dev.yml maintained by Intern 5; all integration testing happens inside it
Late defect discovery on Day 6–7 with no buffer	Medium	Continuous QA involvement from Day 3 onward rather than testing only at the end
Team accidentally starts production-deployment work	Low	Reminder in every stand-up that deployment is out of scope for Milestone 1 and reserved for after Milestone 4

15. Tools & Technology Stack (Milestone 1 Scope)

Layer	Technology
Backend	Python, FastAPI / Django REST Framework
Frontend	React.js, Tailwind CSS
Authentication	JWT, OAuth2
Database	PostgreSQL (relational), Redis (cache)
AI/ML (Milestone-1 scope)	pandas, scikit-learn, spaCy, pdfplumber (EDA + heuristic ATS/recommendation logic only)
API Testing	Postman, pytest
Version Control	Git + GitHub
CI	GitHub Actions (lint, build, test — no deploy stage)

Layer	Technology
Local Integration Environment	Docker & Docker Compose (development use only)
Code Quality	ESLint, Flake8 / Black

16. Sign-Off

This section is completed at the end of Day 7, once the Milestone 1 Acceptance Criteria (Section 13) have been verified against the working increment and the QA sign-off report (Section 11.2 / Section 10 — Intern 5) has been reviewed.

Role	Name	Signature	Date
Intern 1 — Team Lead / Backend & Identity Management			
Intern 2 — Job & Data Intelligence			
Intern 3 — AI/ML Career Intelligence			
Intern 4 — Frontend & User Experience			
Intern 5 — Platform, Testing & Deployment			
Educator / Mentor (Milestone approval)			

Sign-off confirms: all Section 13 acceptance criteria are met; the Section 12 deliverables checklist is complete; no production-deployment activity has occurred; and the team is cleared to begin Milestone 2 planning.